

Word vectors

Hakyung Sung
University of Hawaii at Manoa

September 23, 2026

Topics to cover

- Review
- Distributional semantics
- Vector semantics
- Word2Vec
- Presentation 1: Lu & Hu (2021)
- Presentation 2: Hemilton et al. (2016)

Review: Unit of analysis

Unit of analysis

- The first decision in processing corpus: What counts as a unit?
- Language data are nested
- The choice of lexical unit matters
- BEP tokens are statistical units

The first decision in processing corpus

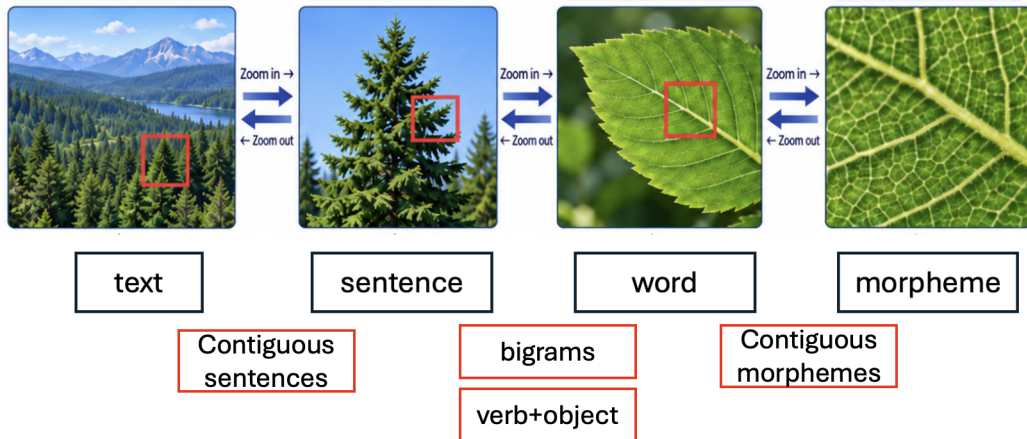


- How should I process my sample?
- In other words, what counts as **one observation**?
- Once we choose a unit of analysis, **how do we define and count it**?

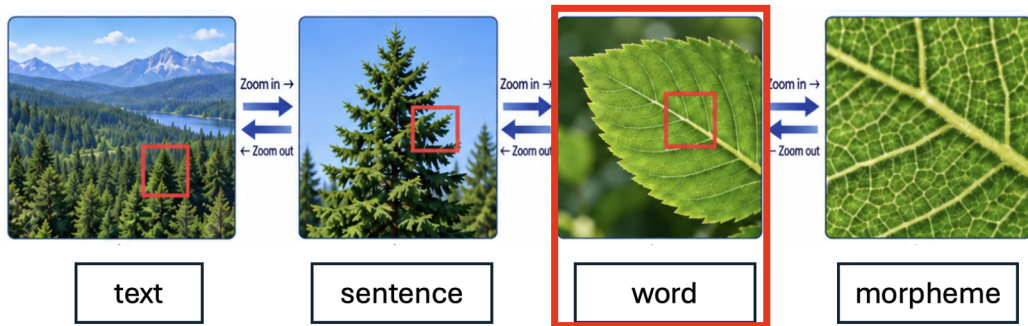
Unit of analysis

- The first decision in processing corpus
- Language data are nested
- The choice of lexical unit matters
- BEP tokens are statistical units

Language data are nested



Language data are nested: Focus on word



Unit of analysis

- The first decision in processing corpus
- Language data are nested
- The choice of lexical unit matters
- BEP tokens are statistical units

Ways of operationalizing “word”, Webb (2021)

Unit	What is grouped together?
Orthographic form (word type)	Each distinct surface form is treated as a separate unit.
Lemma	A headword and its inflectional forms within the same part of speech.
Flemma	A headword and its inflectional forms, without distinguishing part of speech.
Word family	A headword together with its inflectional and derivationally related forms.

The choice of lexical unit matters, Webb (2021)

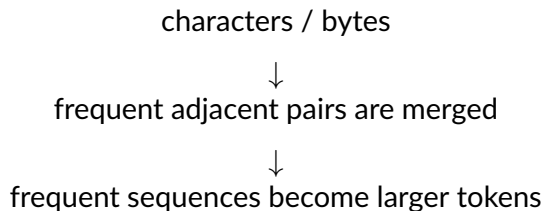
- e.g., I assume that knowledge of one word form implies knowledge of its related inflected or derived forms (e.g., knowing *develop* implies knowing *developed* and *development*)
- Related study:
The decision influences lexical diversity calculations (Jarvis & Hashimoto, 2021)

Unit of analysis

- The first decision in processing corpus
- Language data are nested
- The choice of lexical unit matters
- Modern NLP: Looking for statistical units

Byte-Pair Encoding (BPE): Basic idea

BPE learns subword units from frequency patterns.



e.g.,

$l + o \rightarrow lo \quad \rightarrow \quad lo + w \rightarrow low$

BPE learns statistical units, not “linguistic” units (cf. could be cognitive units; Beinborn & Pinter, 2023).

Introduction

Four weeks ago, we started from...

NLP is a subfield of computer science and artificial intelligence that develops computational methods for enabling **computers to understand, interpret, and generate human language.**

- What does it mean to **understand** human language?
- Understanding language goes beyond recognizing its **form** (e.g., tokens).
- How do we represent **meanings** in computer?

Distributional semantics

Two viewpoints on meaning

There are different theoretical views on what it means to “know” the meaning of a word (Manning, 2022).

- 1. **Denotational semantics:** Meaning is understood in terms of the objects or situations in the world that a linguistic expression refers to or describes.
 - e.g., *empty* describes a state in which something contains none of the contents expected in that context (e.g., an empty glass).

Two viewpoints on meaning

There are different theoretical views on what it means to “know” the meaning of a word (Manning, 2022).

- **2. Distributional semantics:** Meaning is characterized through patterns of linguistic contexts in which an expression appears.
 - e.g., The meaning of *empty* can be characterized by the words and expressions it frequently occurs with:
 - an empty glass / room / container
 - completely empty / almost empty

Representing lexical meaning in NLP

In traditional NLP, words were often represented as discrete symbols.

WORDS AS UNIQUE IDs

Each word is assigned a specific number, like an entry in a dictionary.



ID
12



ID
45



ID
103

NO SENSE OF MEANING

The model has no information that 'cat' and 'dog' are more similar than 'cat' and 'apple'.

CAT vs DOG (ID 12 vs ID 45)



CAT vs APPLE (ID 12 vs ID 103)



Knowledge-based lexical semantics

- **Knowledge-based lexical semantics:** Represent word meanings through explicitly defined concepts and semantic *relationships*.
- Example:
 - WordNet
 - Open English Wordnet
- Drawback: Constructing/maintaining such resources requires substantial human effort.

To distributional semantics

Can we learn semantic relationships directly from patterns of language use?

- Words that occur in similar linguistic contexts tend to have similar meanings.
- Instead of manually specifying semantic relationships, we can estimate them from corpus (again *statistically!*)

Vector semantics

Few related terms

- Words cannot be given as inputs to computers, but numbers can.
- **Numbers** can be.
- **Encoding**: converting words to numbers
- **Vector**: an ordered list of numbers (e.g., [0.1, 0.3, -0.5])
- **One-hot encoding**: a simple encoding

One-hot encoding

- *The cat sat*

word	encoding
the	[1, 0, 0]
cat	[0, 1, 0]
sat	[0, 0, 1]

- Only the entry for the word is set to 1.

One-hot encoding: Problem

- *The cat sat*

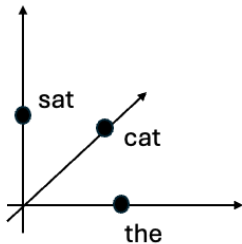
word	encoding
the	[1, 0, 0]
cat	[0, 1, 0]
sat	[0, 0, 1]

- Only the entry for the word is set to 1 (sparse)
- Vectors could be high-dimensional as the vocabulary size

One-hot encoding: Problem

The cat sat

Word	Encoding
the	[1, 0, 0]
cat	[0, 1, 0]
sat	[0, 0, 1]



- One-hot vectors encode word identity, but not semantic relationships.
- **Cosine similarity:**

$$\cos(\theta) = \frac{\vec{A} \cdot \vec{B}}{\|\vec{A}\| \|\vec{B}\|} = \frac{\sum_{i=1}^n A_i B_i}{\sqrt{\sum_{i=1}^n A_i^2} \sqrt{\sum_{i=1}^n B_i^2}}$$

$$\begin{aligned} \text{e.g., } \text{sim}(\text{the}, \text{cat}) &= \frac{(1)(0) + (0)(1) + (0)(0)}{\sqrt{1^2 + 0^2 + 0^2} \sqrt{0^2 + 1^2 + 0^2}} \\ &= \frac{0}{1 \times 1} = 0 \end{aligned}$$

► How to interpret?

One-hot encoding: Solution

- Move from sparse to *distributed* representation
- Vector semantics: Represent word meanings as vectors in a multidimensional space.
- **Word embeddings:** Dense vector representations learned from patterns of language use.

Meaning and language models

- Hypothesis: If meaning is defined as patterns of use and relations among words, then pretrained language models *do* learn meanings.

*...government debt problems turning into **banking** crises as happened in 2009...*
*...saying that Europe needs unified **banking** regulation to replace the hodgepodge...*
*...India has just given its **banking** system a shot in the arm...*

These **context words** will represent **banking**

- “You shall know a word by the company it keeps” (Firth, 1957): One of the most successful ideas of modern statistical NLP.

Sourced from: Manning (2022)

Neural Network–Based Models

Neural models learn vectors directly by predicting context words + neural network
(*we will talk about this more in next week's lecture)

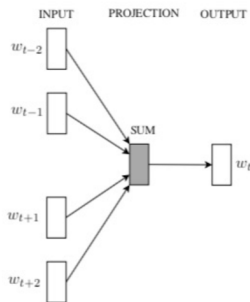
Consistent progress

- 1986: *Learning representations by back propagating errors* (Rumelhart et al., 1986)
- 2003: *A neural probabilistic language model* (Bengio et al., 2003)
- 2013: **Word2Vec** (Skip-gram, CBOW)
- 2014–2015: GloVe, fastText
- 2018– : Contextual embeddings (ELMo, BERT, GPT)

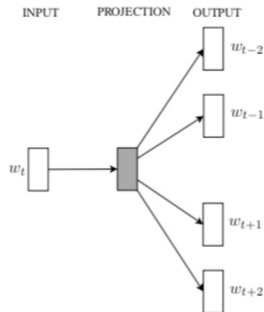
Word2Vec: Overview

- **Word2Vec** (Mikolov et al., 2013) is a framework for learning word embeddings from large corpora.
- The result is a set of **dense word vectors** in which distributionally similar words tend to have similar representations.

Word2Vec: Two models



Continuous Bag of Words (CBOW):
predicting the center words using
the context words ($P(w_t | w_{t-2}, w_{t-1}, w_{t+1}, w_{t+2})$)



Skip-grams (SG):
predicting the context words using
the center word ($P(w_{t+i} | w_t), i \in \{-2, -1, 1, 2\})$)

- CBOW: Predict the center word from its surrounding context.
- **Skip-gram**: Predict surrounding context words from the center word.

Skip-gram: Basic idea

Consider a word occurring in a sentence:

The small **cat** sleeps quietly.

- Choose a **center word**: *cat*
- Define a window around it
- Treat nearby words as its **context words**
- Train the model to predict these context words from the center word

Word2Vec: Skip-grams (*window size* = 1)

- “king brave man”
- “queen beautiful woman”

word	neighbor
king	brave
brave	king
brave	man
man	brave
queen	beautiful
beautiful	queen
beautiful	woman
woman	beautiful

Word2Vec: Skip-grams (*window size* = 2)

- “king brave man”
- “queen beautiful woman”

word	neighbor
king	brave
king	man
brave	man
brave	king
man	king
man	brave
queen	beautiful
queen	woman
beautiful	queen
beautiful	woman
woman	queen
woman	beautiful

Word2Vec: Skip-grams (window size = 2)

word	one-hot encoding	neighbor	one-hot encoding
king	[1, 0, 0, 0, 0, 0]	brave	[0, 1, 0, 0, 0, 0]
king	[1, 0, 0, 0, 0, 0]	man	[0, 0, 1, 0, 0, 0]
brave	[0, 1, 0, 0, 0, 0]	man	[0, 0, 1, 0, 0, 0]
brave	[0, 1, 0, 0, 0, 0]	king	[1, 0, 0, 0, 0, 0]
man	[0, 0, 1, 0, 0, 0]	king	[1, 0, 0, 0, 0, 0]
man	[0, 0, 1, 0, 0, 0]	brave	[0, 1, 0, 0, 0, 0]
queen	[0, 0, 0, 1, 0, 0]	beautiful	[0, 0, 0, 0, 1, 0]
queen	[0, 0, 0, 1, 0, 0]	woman	[0, 0, 0, 0, 0, 1]
beautiful	[0, 0, 0, 0, 1, 0]	queen	[0, 0, 0, 1, 0, 0]
beautiful	[0, 0, 0, 0, 1, 0]	woman	[0, 0, 0, 0, 0, 1]
woman	[0, 0, 0, 0, 0, 1]	queen	[0, 0, 0, 1, 0, 0]
woman	[0, 0, 0, 0, 0, 1]	beautiful	[0, 0, 0, 0, 1, 0]

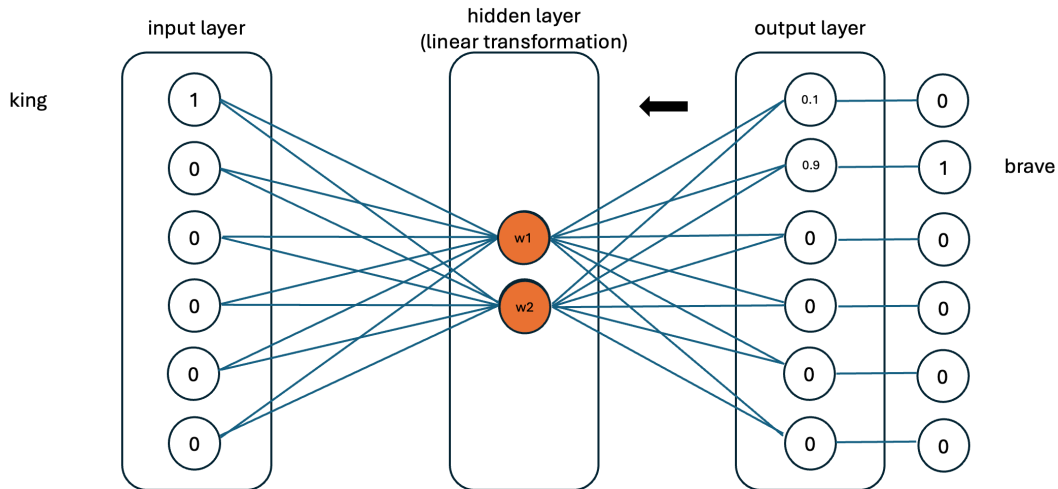
Word2Vec: Input and output

input
[1, 0, 0, 0, 0, 0]
[1, 0, 0, 0, 0, 0]
[0, 1, 0, 0, 0, 0]
[0, 1, 0, 0, 0, 0]
[0, 0, 1, 0, 0, 0]
[0, 0, 1, 0, 0, 0]
[0, 0, 0, 1, 0, 0]
[0, 0, 0, 1, 0, 0]
[0, 0, 0, 0, 1, 0]
[0, 0, 0, 0, 1, 0]
[0, 0, 0, 0, 0, 1]

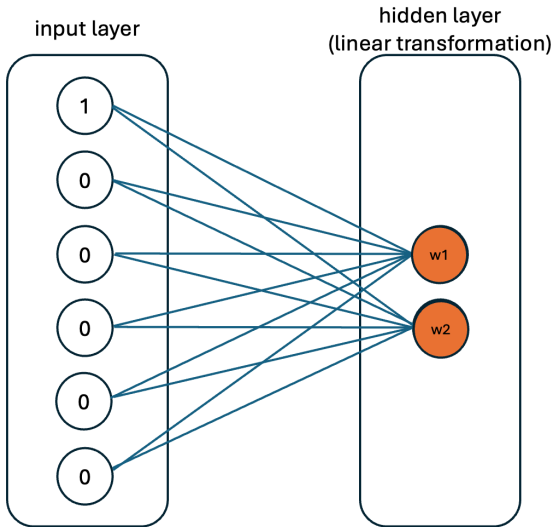
output
[0, 1, 0, 0, 0, 0]
[0, 0, 1, 0, 0, 0]
[0, 0, 1, 0, 0, 0]
[1, 0, 0, 0, 0, 0]
[1, 0, 0, 0, 0, 0]
[0, 1, 0, 0, 0, 0]
[0, 0, 0, 0, 1, 0]
[0, 0, 0, 0, 0, 1]
[0, 0, 0, 1, 0, 0]
[0, 0, 0, 0, 0, 1]
[0, 0, 0, 0, 1, 0]

Word2Vec: Training

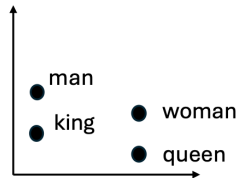
Optimizer:
gradient descent



king



word	embedding
king	[1, 1]
brave	[1, 2]
man	[1, 3]
queen	[5, 1]
beautiful	[5, 2]
woman	[5, 3]



1. From one-hot vectors to word embeddings

- Each word is initially represented as a one-hot vector.
- The input is mapped to a hidden layer through a weight matrix.
- This weight matrix serves as the **embedding matrix**: each row represents a word as a dense vector.
- The weights are learned and updated through the context prediction task.

2. Predict context words

- Given the center word, the model evaluates which words are likely to occur nearby.
- It compares the center word's vector with vectors associated with possible context words.
- More compatible vector representations receive **higher scores**.

3. Learn from positive and negative examples

For **Skip-gram with negative sampling**:

- **Positive examples**: words that actually occur in the center word's context
- **Negative examples**: other words sampled from the vocabulary
- Example:
 - Center: *cat*
 - Positive: *sleeps*
 - Negative: *computer*
- The model learns to give higher scores to observed word-context pairs and lower scores to negative pairs.

4. Update the word vectors

- When the model makes poor predictions, the relevant word vectors are adjusted.
- This process is repeated across many words and contexts in the corpus.
- Words occurring in similar contexts receive similar learning signals.
- As a result, their vector representations tend to become more similar.

5. The resulting embeddings

- After training, each word has a learned **embedding**.
- The location of a word in the vector space reflects patterns of its linguistic use.
- Distributionally similar words tend to be located closer together in the vector space.

Demo

- word2vec <https://projector.tensorflow.org/>
- CosignSimilarity https://editor.p5js.org/_annalena/full/FQmm8akQj
- Applications to language research?

Lu & Hu (2021)

Hemilton et al. (2016)

Appendix

Interpreting cosine similarity

Cosine similarity measures the similarity between two vectors based on the angle between them.

$$\cos(\theta) = \frac{\vec{A} \cdot \vec{B}}{\|\vec{A}\| \|\vec{B}\|}$$

Similarity	Geometric interpretation	Interpretation in word embeddings
1	Same direction (0°)	Maximally similar in vector direction
0	Orthogonal (90°)	No similarity in vector direction
-1	Opposite directions (180°)	Maximally opposite in vector direction